

Chocolate-Doom Telemetry & UX Database

Proyecto de Semestre DBS — Reporte Parte C

Persona 3: Consultas de Sesión · Pipeline ETL · Reproducibilidad

Autores	Sara Coy
	Juan Pablo Vanegas
	Juan Jose Rocha
Consultas	Q1 — Duracion promedio de sesion por mapa
	Q6 — Tics en que jugadores compartieron sector
Entregables	queries_persona3.sql · Makefile · Diccionario de Datos · Muestra TSV
Prerrequisitos	Sql_Entrega_B.sql -> data.sql -> etl3.sql
Fecha	Junio 2026

1. Diccionario de Datos

A continuación se documenta cada columna de las 11 tablas del esquema definido en **Sql_Entrega_B.sql**. Las tablas se presentan en orden de dependencia (padres antes que hijos). Códigos de restricción: PK = llave primaria, FK = llave foránea, UQ = unicidad, CK = check, DEF = valor por defecto.

Tabla: episodio

Columna	Tipo	Nul o	Restricción	Descripción
id_episodio	SERIAL	NO	PK	Llave sustituta autoincremental.
nombre_episodio	VARCHAR(80)	NO	UQ NOT NULL	Nombre del episodio, ej. "Knee-Deep in the Dead". Único en la tabla.

Tabla: mapa

Columna	Tipo	Nul o	Restricción	Descripción
id_mapa	SERIAL	NO	PK	Llave sustituta.
id_episodio	INT	NO	FK -> episodio	Episodio padre.
codigo_mapa	VARCHAR(10)	NO	UQ(ep,code)	Código del motor, ej. E1M1. Único por episodio.
nombre_mapa	VARCHAR(80)	YES	—	Nombre descriptivo, ej. "Hangar".

Tabla: sector

Columna	Tipo	Nul o	Restricción	Descripción
id_sector	SERIAL	NO	PK	Llave sustituta.
id_mapa	INT	NO	FK -> mapa	Mapa al que pertenece el sector.
codigo_sector	VARCHAR(20)	NO	UQ(map,code)	Código del sector, ej. SEC_START. Único por mapa.
coordenada_x	NUMERIC(10,2)	YES	—	Coordenada X de referencia del sector (unidades de juego).
coordenada_y	NUMERIC(10,2)	YES	—	Coordenada Y de referencia del sector (unidades de juego).

Tabla: usuario

Columna	Tipo	Nul o	Restricción	Descripción
id_usuario	SERIAL	NO	PK	Llave sustituta.
codigo_anonimo	VARCHAR(32)	NO	UQ NOT NULL	Hash anonimizado. Nunca almacena nombre real.
edad	INT	YES	CK 10-99	Edad del voluntario en años.
genero	VARCHAR(20)	YES	—	Género autoreportado.

Columna	Tipo	Nul o	Restricción	Descripción
experiencia	VARCHAR(20)	YES	—	Nivel de experiencia en videojuegos autoreportado.
consentimiento	BOOLEAN	NO	DEF FALSE	Consentimiento informado. El ETL rechaza datos si es FALSE.
fecha_registro	TIMESTAMP	NO	NOT NULL	Marca temporal del registro y consentimiento.

Tabla: jugador

Columna	Tipo	Nul o	Restricción	Descripción
id_jugador	SERIAL	NO	PK	Llave sustituta.
alias	VARCHAR(50)	NO	UQ(usr,alias)	Alias en el juego. Único por usuario.
id_usuario	INT	NO	FK -> usuario	Usuario voluntario propietario del alias.

Tabla: partida

Columna	Tipo	Nul o	Restricción	Descripción
id_partida	SERIAL	NO	PK	Llave sustituta.
id_mapa	INT	NO	FK -> mapa	Mapa jugado en la sesión.
fecha_inicio	TIMESTAMP	NO	NOT NULL	Marca temporal de inicio de la sesion.
fecha_fin	TIMESTAMP	YES	CK fin>=ini	Fin de la sesión. NULL si la partida está en curso.
configuracion	TEXT	YES	—	Cadena de configuración: dificultad, motor de render, etc.

Tabla: participante_partida

Columna	Tipo	Nul o	Restricción	Descripción
id_participante	SERIAL	NO	PK	Llave sustituta.
id_partida	INT	NO	FK -> partida	Sesión a la que se unió el jugador.
id_jugador	INT	NO	FK -> jugador UQ(partida,jugador)	Jugador en la sesión. Un jugador por sesión.
rol	VARCHAR(30)	YES	—	Rol asignado, ej. Player_1 / Player_2.

Tabla: evento_telemetria

Columna	Tipo	Nul o	Restricción	Descripción
id_evento	BIGSERIAL	NO	PK	Llave sustituta autoincremental (BIGSERIAL para alta frecuencia).
id_partida	INT	NO	FK -> partida	Sesión a la que pertenece el evento.

Columna	Tipo	Nul o	Restricción	Descripción
id_jugador	INT	NO	FK -> jugador	Jugador que genero el evento.
id_sector	INT	YES	FK -> sector	Sector en el momento del tic. NULL si no fue registrado.
tic	INT	NO	CK >=0 UQ(partida,jugador,tic)	Contador de tic del motor. Único por jugador por sesión.
pos_x	NUMERIC(12,4)	YES	—	Posición X del jugador en unidades de juego.
pos_y	NUMERIC(12,4)	YES	—	Posición Y del jugador en unidades de juego.
pos_z	NUMERIC(12,4)	YES	—	Posición Z (altura) del jugador en unidades de juego.
angulo	NUMERIC(8,4)	YES	—	Ángulo de visión en grados [0-360).
momentum_x	NUMERIC(10,4)	YES	—	Componente X del vector de movimiento. Negativo = izquierda.
momentum_y	NUMERIC(10,4)	YES	—	Componente Y del vector de movimiento.
momentum_z	NUMERIC(10,4)	YES	—	Componente Z (momentum vertical).
fov	NUMERIC(6,2)	YES	—	Ángulo del campo visual en grados.
salud	INT	YES	CK >=0	Puntos de salud del jugador (0 = muerto).
armadura	INT	YES	CK >=0	Puntos de armadura.
municion	INT	YES	CK >=0	Contador de municion actual.

Tabla: instrumento_ux

Columna	Tipo	Nul o	Restricción	Descripción
id_instrumento	SERIAL	NO	PK	Llave sustituta.
nombre	VARCHAR(50)	NO	UQ NOT NULL	Nombre del instrumento, ej. BANGS, PENS, GUESS.
descripcion	TEXT	YES	—	Descripción completa del instrumento UX.

Tabla: item_ux

Columna	Tipo	Nul o	Restricción	Descripción
id_item	SERIAL	NO	PK	Llave sustituta.
texto_pregunta	VARCHAR(255)	NO	NOT NULL	Texto completo de la pregunta mostrada al voluntario.
dimension	VARCHAR(40)	YES	—	Dimensión BANGS: Boredom / Anxiety / Nausea / Frustration / Game-Satisfaction.
id_instrumento	INT	NO	FK -> instrumento_ux	Instrumento al que pertenece el ítem.

Tabla: respuesta_ux

Columna	Tipo	Nul o	Restricción	Descripción
id_respuesta	SERIAL	NO	PK	Llave sustituta.
fecha_respuesta	TIMESTAMP	NO	NOT NULL	Momento en que el voluntario envió la respuesta.
id_usuario	INT	NO	FK -> usuario	Voluntario que respondió.
id_item	INT	NO	FK -> item_ux	Pregunta respondida.
id_instrumento	INT	NO	FK -> instrumento_ux	Instrumento utilizado (desnormalizado para rendimiento).
id_partida	INT	YES	FK -> partida	Sesión a la que refiere la respuesta. NULL = encuesta post-estudio.
valor_respuesta	NUMERIC(5,2)	NO	NOT NULL	Respuesta en escala Likert, ej. 1.00-5.00.

2. Muestra de Telemetría Cruda (TSV)

El motor Chocolate-Doom modificado emite una línea separada por tabuladores por cada tic. El formato de 15 columnas mostrado a continuación es la entrada que consume la tabla de staging **stg_evento_telemetria**. La fila 8 es intencionalmente incorrecta para demostrar el rechazo del pipeline ETL.

Columnas 1–8:

id_partida a	id_jugador r	tic	pos_x	pos_y	pos_z	angulo	momentum_ x
1	1	1	0.0000	150.0000	0.0000	180.0000	5.0000
1	1	2	9.9833	149.8612	0.0200	181.7952	4.9601
1	1	3	19.8669	149.4451	0.0600	183.5905	4.8413
1	2	1	0.0000	150.0000	0.0000	180.0000	5.0000
1	2	2	9.9833	149.8612	0.0200	181.7952	4.9601
2	3	1	0.0000	150.0000	0.0000	180.0000	5.0000
2	3	2	9.9833	149.8612	0.0200	181.7952	4.9601
99	99	-1	ABC	150.0	0.0	180.0	5.0

Columnas 9–15:

momentum_ y	momentum_ z	fov	salud	armadura	municion	id_sector	
0.0000	0.0000	90.00	80	100	90	1	
0.3987	0.0000	90.00	80	100	90	1	
0.7946	0.0000	90.00	80	100	91	1	
0.0000	0.0000	90.00	80	100	90	1	
0.3987	0.0000	90.00	79	98	88	1	
0.0000	0.0000	90.00	80	100	90	1	
0.3987	0.0000	90.00	80	99	85	1	
0.0	0.0	90.00	80	100	90	1	* malformada

Fila 8 (malformada): id_partida=99 (FK invalida), id_jugador=99 (FK invalida), tic=-1 (falla el check de rango), pos_x="ABC" (falla el check de tipo). Los pasos 1 a 4 del ETL capturan esta fila en log_errores_carga — nunca llega a evento_telemetria.

Comando de carga (psql interactivo):

```
\copy stg_evento_telemetria (  
  raw_id_partida, raw_id_jugador, raw_tic,  
  raw_pos_x, raw_pos_y, raw_pos_z, raw_angulo,  
  raw_momentum_x, raw_momentum_y, raw_momentum_z,  
  raw_fov, raw_salud, raw_armadura, raw_municion, raw_id_sector  
)  
FROM '/ruta/al/archivo/telemetria.tsv'  
WITH (FORMAT text, DELIMITER E'\t', HEADER true);
```

3. Descripción del Pipeline ETL

El pipeline en **etl3.sql** mueve la telemetría cruda desde la tabla de staging hacia la tabla core **evento telemetria** en ocho pasos de validación ordenados, todos dentro de una transacción atómica (BEGIN / COMMIT). Cada registro rechazado se escribe en **log_errores_carga** con su motivo, garantizando trazabilidad completa de auditoría.

Paso	Validación	Lógica
Paso 1	Campos obligatorios nulos o vacíos	Rechaza filas donde id_partida, id_jugador o tic son NULL o vacíos.
Paso 2	Tipos de datos inválidos	Valida todos los campos numéricos con expresiones regulares. Los campos opcionales (ángulo, fov, id_sector) solo se validan si vienen con valor.
Paso 3	Valores fuera de rango	Rechaza tic < 0, salud < 0, armadura < 0 o munición < 0.
Paso 4a	FK id_jugador inválida	Rechaza filas que referencian un jugador inexistente.
Paso 4b	FK id_partida inválida	Rechaza filas que referencian una sesión inexistente.
Paso 4c	Sector fuera del mapa	Si id_sector está informado, verifica que pertenezca al mapa de la sesión referenciada.
Paso 5	Sin consentimiento	Rechaza telemetría de usuarios con consentimiento = FALSE (requisito ético).
Paso 6	Duplicados en el lote	ROW_NUMBER() OVER (PARTITION BY partida,jugador,tic) conserva solo la primera ocurrencia del lote.
Paso 7	Duplicados contra el core	Registra filas cuya clave (id_partida, id_jugador, tic) ya existe en evento_telemetria.
Paso 8	INSERT de registros válidos	Inserta todas las filas que pasaron los filtros anteriores. ON CONFLICT DO NOTHING como segunda barrera.

Consulta de verificación post-carga:

```
SELECT motivo, COUNT(*) AS cantidad
FROM   log_errores_carga
GROUP  BY motivo
ORDER  BY cantidad DESC;

SELECT COUNT(*) AS total_core FROM evento_telemetria;
```

4. Consulta Q1 — Duración Promedio de Sesiones por Mapa

Objetivo: Identificar qué mapas generan sesiones de juego más largas o cortas en promedio. Esto permite formular hipótesis sobre la complejidad del mapa y el nivel de involucramiento del jugador.

SQL:

```
SELECT
  e.nombre_episodio                AS episodio,
  m.codigo_mapa                    AS mapa,
  m.nombre_mapa                    AS nombre_mapa,
  COUNT(p.id_partida)              AS total_partidas,
  ROUND(AVG(
    EXTRACT(EPOCH FROM (p.fecha_fin - p.fecha_inicio)) / 60.0
  ))::numeric, 2)                  AS duracion_promedio_min,
  ROUND(MIN(
    EXTRACT(EPOCH FROM (p.fecha_fin - p.fecha_inicio)) / 60.0
  ))::numeric, 2)                  AS duracion_minima_min,
  ROUND(MAX(
    EXTRACT(EPOCH FROM (p.fecha_fin - p.fecha_inicio)) / 60.0
  ))::numeric, 2)                  AS duracion_maxima_min
FROM partida p
JOIN mapa m ON m.id_mapa = p.id_mapa
JOIN episodio e ON e.id_episodio = m.id_episodio
WHERE p.fecha_fin IS NOT NULL
GROUP BY e.id_episodio, e.nombre_episodio,
         m.id_mapa, m.codigo_mapa, m.nombre_mapa
ORDER BY e.id_episodio, duracion_promedio_min DESC;
```

Resultados:

Episodio	Mapa	Nombre	Partidas	Prom. (min)	Mín (min)	Máx (min)
Knee-Deep in the Dead	E1M1	Hangar	1	25.00	25.00	25.00
The Shores of Hell	E2M1	Deimos Lab	1	40.00	40.00	40.00
Inferno	E3M1	Hell Keep	1	15.00	15.00	15.00

Interpretación: Deimos Lab (40 min, Ultra-Violence) tuvo la sesión más larga; Hell Keep (15 min, Nightmare) la más corta, consistente con el nivel de dificultad configurado. Con más sesiones por mapa, el promedio revelaría diferencias estadísticamente significativas entre mapas.

Evaluación de índices — Q1:

Q1 opera sobre **partida**, **mapa** y **episodio**, todas tablas pequeñas (3 filas cada una en el dataset de prueba). PostgreSQL elige correctamente **Seq Scan** en ambas ejecuciones; los index scans no son beneficiosos por debajo de ~100 filas. El plan y el tiempo de ejecución son idénticos antes y después de crear los índices, confirmando que la consulta ya es óptima para este volumen de datos.

ANTES de índices — Tiempo de ejecución: 0.197 ms

```
Sort (cost=163.91..166.32 rows=965 width=506)
  (actual time=0.090..0.092 rows=3 loops=1)
  Sort Key: e.id_episodio, duracion_promedio_min DESC
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=9
-> HashAggregate (actual time=0.060..0.068 rows=3 loops=1)
  -> Hash Join Hash Cond: (m.id_episodio = e.id_episodio)
    -> Hash Join Hash Cond: (p.id_mapa = m.id_mapa)
      -> Seq Scan on partida p
        Filter: (fecha_fin IS NOT NULL)
        Buffers: shared hit=1
      -> Hash on mapa m Buffers: shared hit=1
    -> Hash on episodio e Buffers: shared hit=1
```


Planning Time: 0.819 ms | Execution Time: 0.197 ms

DESPUÉS de índices — Tiempo de ejecución: 0.213 ms

```
Sort (cost=163.91..166.32 rows=965 width=506)
  (actual time=0.112..0.114 rows=3 loops=1)
  Sort Key: e.id_episodio, duracion_promedio_min DESC
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=9
-> HashAggregate (actual time=0.060..0.068 rows=3 loops=1)
  -> Hash Join Hash Cond: (m.id_episodio = e.id_episodio)
    -> Hash Join Hash Cond: (p.id_mapa = m.id_mapa)
      -> Seq Scan on partida p
        Filter: (fecha_fin IS NOT NULL)
        Buffers: shared hit=1
      -> Hash on mapa m Buffers: shared hit=1
    -> Hash on episodio e Buffers: shared hit=1
Planning Time: 0.580 ms | Execution Time: 0.213 ms
```

Análisis: Sin cambio medible (0.197 ms vs 0.213 ms — dentro del margen de ruido). Q1 no accede a evento_telemetria, por lo que ninguno de los tres índices creados aplica. Los índices fueron diseñados para acelerar Q2, Q3, Q6, Q7 y Q8, que realizan self-joins y escaneos de rango sobre las 21,600 filas de telemetría.

5. Consulta Q6 — Tics en que Jugadores Compartieron Sector

Objetivo: Cuantificar la co-presencia entre pares de jugadores dentro del mismo sector del mapa. Una alta co-presencia es el indicador primario de cooperación; co-presencia sostenida en sectores con alta densidad de enemigos puede señalar sabotaje.

SQL:

```
SELECT
  a.id_partida,
  e.nombre_episodio,
  m.codigo_mapa,
  s.codigo_sector,
  ja.alias,
  jb.alias,
  COUNT(DISTINCT a.tic)
  ROUND(
    COUNT(DISTINCT a.tic)::numeric /
    NULLIF(
      (SELECT COUNT(DISTINCT tic)
       FROM evento_telemetria
       WHERE id_partida = a.id_partida), 0
    ) * 100.0, 2)
  AS pct_partida_juntos
FROM evento_telemetria a
JOIN evento_telemetria b
  ON a.id_partida = b.id_partida
  AND a.tic = b.tic
  AND a.id_sector = b.id_sector
  AND a.id_jugador < b.id_jugador
JOIN sector s ON s.id_sector = a.id_sector
JOIN partida p ON p.id_partida = a.id_partida
JOIN mapa m ON m.id_mapa = p.id_mapa
JOIN episodio e ON e.id_episodio = m.id_episodio
JOIN jugador ja ON ja.id_jugador = a.id_jugador
JOIN jugador jb ON jb.id_jugador = b.id_jugador
GROUP BY a.id_partida, e.nombre_episodio, m.codigo_mapa,
         s.codigo_sector, ja.alias, jb.alias
ORDER BY a.id_partida, tics_compartidos DESC;
```

Resultados:

Sesión	Episodio	Mapa	Sector	Jugador A	Jugador B	Tics compart.	% sesión
1	Knee-Deep...	E1M1	SEC_START	DoomSlayer_99	Sara_Cyborg	1200	100.00
2	Shores of Hell	E2M1	SEC_START	Juanpa_Master	NoobMarine	1200	100.00
3	Inferno	E3M1	SEC_START	CyberDemon_Hunter	John_Romero_Fan	1200	100.00

Nota sobre datos sintéticos: *data.sql* asigna `id_sector=1 (SEC_START)` a todos los eventos, por lo que el 100% de co-presencia es esperado. Con telemetría real del motor, los `id_sector` variarían y los porcentajes diferenciarían patrones de cooperación por sector.

Evaluación de índices — Q6:

Q6 realiza un self-join sobre **21,600 filas** de `evento_telemetría` igualando (`id_partida`, `tic`, `id_sector`), más una subconsulta correlacionada por cada grupo de salida. Es la consulta más costosa computacionalmente del proyecto.

ANTES de índices — Tiempo de ejecución: 608.227 ms

```
Incremental Sort (actual time=607.273..607.283 rows=45 loops=1)
  Buffers: shared hit=663,984
  -> GroupAggregate (actual time=180.449..607.206 rows=45 loops=1)
```

```

-> Nested Loop (actual time=21.083..304.095 rows=54,000 loops=1)
  -> Merge Join en (partida, sector, tic)
      Filtro: (a.id_jugador < b.id_jugador)
      Filas eliminadas: 75,600
      -> Sort + Seq Scan en evento_telemetria a
          filas=21,600 Memoria: 1612kB
      -> Sort + Seq Scan en evento_telemetria b
          filas=21,600 Memoria: 1612kB
      -> Index Scan en sector_pkey (53,999 hits en cache)
SubPlan (pct_partida_juntos):
  -> Bitmap Heap Scan loops=45 filas=7,200/loop
      Bitmap Index Scan en restricción UNIQUE
Planning Time: 3.286 ms | Execution Time: 608.227 ms

```

DESPUÉS de índices — Tiempo de ejecución: 1062.299 ms (JIT: +666 ms)

```

Incremental Sort (actual time=1020.156..1020.177 rows=45 loops=1)
  Buffers: shared hit=15,772 read=84
  -> GroupAggregate (actual time=768.946..1020.076 rows=45 loops=1)
      -> Merge Join en a.id_partida = p.id_partida
          -> Merge Join en (partida, sector, tic)
              Filtro: (a.id_jugador < b.id_jugador)
              Filas eliminadas: 75,600
              -> Sort + Seq Scan en evento_telemetria a
                  filas=21,600 Memoria: 1978kB
              -> Sort + Seq Scan en evento_telemetria b
                  filas=21,600 Memoria: 1978kB
          -> Memoize en sector_pkey
              Cache Hits: 53,999 / Misses: 1
SubPlan (pct_partida_juntos):
  -> Bitmap Heap Scan loops=45
      Bitmap Index Scan en idx_tel_partida_jugador_tic (NUEVO)
          filas=7,200/loop Buffers leídos=84 (I/O disco)
JIT: Generación 9ms | Inlining 99ms | Optimización 352ms | Emisión 205ms
Planning Time: 14.118 ms | Execution Time: 1062.299 ms

```

Análisis: La aparente desaceleración (608 ms -> 1062 ms) se explica completamente por el **overhead de compilación JIT (+666 ms)**: PostgreSQL activó JIT por primera vez al ver que las nuevas estadísticas de los índices elevaron el estimado de filas de 7 a 43,200. Este costo es de una sola vez; en un servidor con planes en caché no volvería a ocurrir. El índice **idx_tel_partida_jugador_tic** Sí se utiliza en la subconsulta (bitmap scan, 84 lecturas de disco vs. 1,455 shared hits antes), confirmando su efectividad para esa operación. El self-join sigue usando Seq Scan porque con 21,600 filas el planificador prefiere ordenamiento sobre acceso por índice. A escala de producción (millones de filas), el índice también aceleraría el self-join.

6. Reproducibilidad — Makefile

Un **Makefile** en la raíz del proyecto proporciona un mecanismo de un solo comando para recrear el esquema completo, cargar datos, ejecutar el pipeline ETL y verificar conteos de filas, cumpliendo el requisito de reproducibilidad de la Parte C.

Archivo: Makefile (colocar en la raíz del proyecto junto a todos los .sql)

```
DB_URL ?= postgresql://postgres:postgres@localhost:5432/chocolate_doom

.PHONY: all recreate load verify clean

## Configuración completa en un solo comando
all: recreate load verify

## Elimina y recrea el esquema
recreate:
■@echo "Recreando esquema..."
■psql "$(DB_URL)" -f Sql_Entrega_B.sql
■@echo "Esquema listo."

## Carga datos sintéticos y ejecuta ETL
load:
■@echo "Cargando datos (data.sql)..."
■psql "$(DB_URL)" -f data.sql
■@echo "Ejecutando ETL (etl3.sql)..."
■psql "$(DB_URL)" -f etl3.sql
■@echo "Carga completa."

## Muestra conteos de filas por tabla
verify:
■psql "$(DB_URL)" -c "\
■SELECT 'evento_telemetria' AS tabla, COUNT(*) AS filas \
■FROM evento_telemetria \
■UNION ALL SELECT 'jugador', COUNT(*) FROM jugador \
■UNION ALL SELECT 'usuario', COUNT(*) FROM usuario \
■UNION ALL SELECT 'partida', COUNT(*) FROM partida \
■UNION ALL SELECT 'respuesta_ux', COUNT(*) FROM respuesta_ux \
■UNION ALL SELECT 'log_errores_carga', COUNT(*) FROM log_errores_carga \
■ORDER BY tabla;"

## Elimina todo (usar antes de recreate en una BD sucia)
clean:
■psql "$(DB_URL)" -c \
■ "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
■@echo "Esquema eliminado."
```

Comando	Efecto
make all	Pipeline completo: recreate -> load -> verify
make recreate	Elimina todas las tablas y recrea el esquema desde Sql_Entrega_B.sql
make load	Ejecuta data.sql y luego etl3.sql
make verify	Imprime conteos de filas para todas las tablas principales
make clean	DROP SCHEMA public CASCADE — usar con precaución

Prerrequisitos: PostgreSQL 14+, cliente psql, make. Configurar DB_URL como variable de entorno o editar el valor por defecto en el Makefile. La base de datos debe existir antes de ejecutar make recreate.

7. Conclusiones

Diseño del esquema y calidad de datos

El esquema relacional desarrollado en este proyecto modela de forma correcta y completa el dominio de telemetría de Chocolate-Doom. Las 11 tablas cubren todas las entidades relevantes — estructura de niveles (episodio, mapa, sector), voluntarios (usuario, jugador), sesiones de juego (partida, participante_partida), eventos de telemetría de alta frecuencia y el instrumento UX BANGS —, con llaves primarias, foráneas, restricciones de unicidad y checks de dominio que garantizan integridad referencial en todo momento. El pipeline ETL de ocho pasos demostró ser robusto: valida tipos, rangos, llaves foráneas, consentimiento ético y duplicados antes de insertar cualquier registro en el core, rechazando correctamente la fila malformada incluida en la muestra TSV.

Consultas analíticas y window functions

Las consultas Q1 y Q6 cubren dos dimensiones complementarias del objetivo de investigación. Q1 revela diferencias en duración de sesión por mapa: Deimos Lab (40 min, Ultra-Violence) supera a Hell Keep (15 min, Nightmare), lo cual es consistente con el nivel de dificultad y sugiere que mapas más difíciles generan sesiones más cortas por muertes frecuentes. Q6 cuantifica la co-presencia entre pares de jugadores, introduciendo la columna **pct_partida_juntos** que normaliza los tics compartidos por la duración total de la sesión, permitiendo comparaciones entre partidas de distinta longitud. Otras consultas del grupo (Q3, Q7) hacen uso de window functions LAG() y RANK(), activando el criterio de bonificación del enunciado.

Evaluación de índices y rendimiento

Se crearon tres índices sobre evento_telemetria siguiendo las sugerencias del enunciado: un índice compuesto en (id_partida, id_jugador, tic) para self-joins y consultas de trayectoria, un índice en id_sector para consultas de hotspot y co-presencia, y un índice BTREE en (pos_x, pos_y) para consultas de proximidad espacial. El análisis EXPLAIN ANALYZE mostró resultados diferenciados según la naturaleza de cada consulta: Q1 no se beneficia de los índices porque opera sobre tablas de dimensión pequeñas (3 filas), donde el Seq Scan siempre es preferido por el planificador. Q6, en cambio, evidenció que el índice idx_tel_partida_jugador_tic es utilizado en la subconsulta correlacionada (bitmap scan con 84 lecturas de disco confirmadas), reduciendo el acceso aleatorio. La activación de JIT en la segunda ejecución de Q6 agregó 666 ms de overhead de compilación de una sola vez, explicando el tiempo mayor observado; en producción con planes en caché este costo desaparece. A escala real (millones de filas de telemetría), los tres índices serían determinantes para mantener tiempos de respuesta aceptables en todas las consultas analíticas.

Vistas y materialización

Las tres vistas entregadas por el grupo resuelven los casos de uso más frecuentes: **v_trayectoria_por_jugador** precalcula el centroide de movimiento, rango de desplazamiento y estadísticas vitales por sesión; **v_copresencia_sectores** detecta pares de jugadores en el mismo sector y tic, siendo el detector principal de cooperación y sabotaje; y **mv_hotspot_sectores**, como vista materializada con RANK() embebido, permite consultar los sectores más visitados sin reescanear las 21,600 filas de telemetría en cada acceso, siendo ideal para dashboards y reportes recurrentes de investigación.

Reproducibilidad y ética

El Makefile entregado permite recrear el entorno completo en un solo comando (make all), cumpliendo el requisito de reproducibilidad. Desde el punto de vista ético, el esquema implementa anonimización de usuarios mediante código hash (codigo_anonimo), el campo consentimiento controla el flujo de datos en el ETL rechazando cualquier registro de un usuario que no haya otorgado su consentimiento informado, y la tabla log_errores_carga garantiza trazabilidad completa de todos los rechazos para auditoría posterior.